

Joint Privacy-Preserving Cloud Workload Prediction Based on Federated Learning

Bin Yang[✉], *Member, IEEE*, Li Yan[✉], *Member, IEEE*, Zhuozhao Li[✉], *Member, IEEE*, Mingjun Kao[✉],
Huanbo Gao, Xingye Sun, and Chao Shen[✉], *Fellow, IEEE*

Abstract—Organizations around the world are increasingly using multiple clouds for critical workloads, which makes the accurate prediction of cloud workload a collaborative task across multiple cloud providers. Our experimental studies on four large-scale cloud workload datasets demonstrate that the workloads are highly heterogeneous, and different cloud providers have highly heterogeneous model preferences, which makes the above task challenging. Accordingly, we propose a joint privacy-preserving cloud workload prediction framework based on Federated Learning (FL), which enables the collaborative training of a workload prediction model across cloud providers with heterogeneous workloads and model preferences without exposing private workload data. To handle workload heterogeneity without privacy leakage, we first design a Generative Adversarial Network (GAN) based virtual workload dataset generation method that incorporates workloads' temporal and resource-usage-plan-related features via temporal-aware feature calibration. Then, to handle model heterogeneity, we design a FL approach based on a hybrid local model composed of a homogeneous public model and a heterogeneous personal model. Finally, we design a Knowledge Distillation (KD) based approach to post-train the personal model with both private workload knowledge and shared workload knowledge learned from the trained global public model. Extensive experiments on various real-world workloads and actual implementations demonstrate that compared with the state-of-the-art, our framework improves workload prediction accuracy by 57.7% in average over all cloud providers.

Index Terms—Cloud workload prediction, federated learning, knowledge distillation.

I. INTRODUCTION

WITH the popularity of diverse cloud services, the workloads on cloud providers are becoming increasingly

more heterogeneous and dynamic, which causes several problems such as insufficient resource utilization, long queuing delay, etc. [1], [2]. To remedy the problems, a prediction framework that optimally estimates possible fluctuations of incoming workloads is expected [3], [4].

Driven by this expectation, many traditional approaches focusing on the prediction of workloads on a single cloud provider have been proposed [4], [5], [6], [7], [8]. Generally, they apply various kinds of Machine Learning (ML) methodologies (e.g., Recurrent Neural Networks (RNN), Autoregressive Integrated Moving Average (ARIMA)) to learn the high-dimensional patterns in historical workload records, and predict workload values in the near future. However, along with the popularity of dynamic containers and cloud-based applications, the workload patterns are becoming increasingly diverse and difficult for generalization [9], [10], [11], [12], which may cause the isolated workload databases of cloud providers to be biased. For example, several public cloud providers (e.g., *Alibaba*) have abundant data for the analysis of ML workload patterns, but lack sufficient experience for the accurate prediction of scientific computing workloads [13]. As a result, cloud providers - especially newly established ones with limited training data - cannot rely on traditional approaches. These providers need a good cold-start workload prediction model, but traditional methods cannot build a versatile workload predictor capable of accurately predicting workloads covering various categories of services, which are quite common nowadays [3], [14].

In this context, there have been some pioneer works indicating the necessity of establishing a comprehensive workload prediction framework via the collaboration across different cloud providers [3], [15], [16]. Nevertheless, due to the intrinsic heterogeneity and concerns of privacy leakage or trade secrecy, the paradigm of simply aggregating workload data from different cloud providers for the training of a prediction model is not feasible [17]. In a report commissioned by CISCO [18], Eric Hanselman, the Principal Research Analyst, indicated that “each cloud environment can have its own operational model and management environment. Without a common framework to manage them, security teams need to develop fluency in each new cloud – a significant investment in time and resources.” Other organizations (e.g., YouTube, Netflix) have reported similar experiences [19], [20].

Driven by the increased privacy concerns, Federated Learning (FL) was proposed [17], [21]. Different from previous ML based methods that rely on data directly collected from cloud providers, FL enables joint ML model training without exposing privacy-sensitive data. It has been confirmed in several recent works [17], [22], [23] that FL is promising

Received 25 October 2024; revised 15 July 2025; accepted 8 December 2025; approved by IEEE TRANSACTIONS ON NETWORKING Editor A. Khreishah. Date of publication 17 December 2025; date of current version 12 January 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB4500800, in part by the National Natural Science Foundation of China under Grant 62103325 and Grant 62202216, in part by Shaanxi High-Level Talent Program under Grant 2021QCYRC4-26, in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2023A1515010244, and in part by the Shenzhen Science and Technology Program under Grant 20231121101752002. (*Corresponding author: Li Yan.*)

Bin Yang, Li Yan, and Chao Shen are with the School of Cyber Science and Engineering, Xi'an Jiaotong University, Xi'an, Shaanxi 710049, China (e-mail: devilyb@stu.xjtu.edu.cn; li.yan.88@xjtu.edu.cn; chaoshen@mail.xjtu.edu.cn).

Zhuozhao Li is with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: lizz@sustech.edu.cn).

Mingjun Kao, Huanbo Gao, and Xingye Sun are with the Neusoft Group Corporation Ltd., Shenyang, Liaoning 110179, China (e-mail: kaomj@neusoft.com; gao-hb@neusoft.com; sunxingye@neusoft.com).

Data is available on-line at <https://github.com/liyan2015/JC-workload-predictor>

Digital Object Identifier 10.1109/TON.2025.3643456

for realizing privacy-preserving prediction of workloads across multiple cloud providers, but face several challenges:

A. Data Heterogeneity

As demonstrated in several recent works [13], [24] and our experimental studies (Section III), the workloads coming from different cloud providers exhibit heterogeneous characteristics due to their different origins and involvement in diverse business operations, which is known as the Non-IID (Non-Identically and Independently Distributed) issue, and may cause significant performance degradation in FL training [25], [26]. Although there is an urgent demand for many cloud providers, such as *Microsoft* [24] and *Alibaba* [13], to learn as many workload records as possible, the collaborative training of a practical workload prediction model given the highly heterogeneous training data is non-trivial.

B. Model Heterogeneity

Due to various factors (e.g., difference in preference of service, computing infrastructure, etc.), different cloud providers may have quite heterogeneous workload categories, as well as hardware resources and computing capabilities, which leads to their various preferences for the optimal workload prediction models [27], [28]. However, a common and unrealistic assumption in traditional FL is that all participating clients have uniform resource requirements and share an identical global model structure [29], [30]. As a result, the collaborative training of a workload prediction model with various structures and sizes for respective cloud providers is necessary but challenging.

In recent years, there have been multiple works proposed to handle the challenges. For instance, Knowledge Distillation (KD) has been utilized in many works to solve the model heterogeneity issue [31], [32], [33], [34], [35], [36]. Generally, these works use a proxy dataset, which is constructed from partial data collected from each of the participating FL clients, to represent global data distribution. However, these works require the clients to send additional information for KD (e.g., model parameters and temporary output of the intermediate model) during training, which significantly increases communication cost and privacy leakage risk compared to traditional FL methods such as FedAvg [37].

Meanwhile, several works proposed to tackle data heterogeneity issue via privacy-preserving similarity measurement of data distribution and client clustering [38], [39], [40], [41]. In comparison, the method of rectifying the Non-IID data via the generation of virtual data [42], which is called the Virtual Homogeneity Learning (VHL), has been confirmed more effective in improving the convergence speed and privacy protection of FL, while maintaining high generalization performance on both training and testing datasets. Similar to the paradigm of VHL, some recent works [43], [44] proposed to combine virtual data generation with KD for image data augmentation. However, these methods are mostly designed for data with spatial features (e.g., image classification task), and thus cannot be applied to cloud workload time

series data, of which temporal features are crucial for model performance.

Accordingly, we propose a joint privacy-preserving cloud workload prediction framework based on FL, which enables the collaborative training of a workload prediction model across cloud providers with heterogeneous workloads and model preferences without exposing private workload data. Specifically, to handle heterogeneous workloads of different cloud providers without privacy leakage, we first design a Generative Adversarial Network (GAN) based virtual workload dataset generation method that incorporates the temporal and resource-usage-plan-related features of the workloads via temporal-aware feature calibration. Then, to handle heterogeneous model preferences of different cloud providers, we design a FL approach based on a hybrid local model composed of a homogeneous public model for participating in global model aggregation over the generated virtual workload datasets, and a heterogeneous personal model for adapting to respective private workload environment of the cloud providers. Finally, we design a KD based post training approach of personal models to transfer knowledge from the trained global public model to the personal model, and meanwhile use the private workload data to adjust the personal model training direction. In summary, our contributions include:

- 1) We conduct extensive experimental studies on four large-scale cloud workload datasets to demonstrate the heterogeneity of cloud providers' workloads, and deficiencies of existing methods in generating virtual workload data, and potential benefit of users' resource usage plan on improving workload prediction accuracy.
- 2) We propose a novel joint privacy-preserving cloud workload prediction framework. It combines GAN with a novel temporal-aware feature calibration method to generate virtual IID workload datasets, utilizes a FL-based hybrid model training approach to allow heterogeneous model preferences of different cloud providers, and a KD-based approach for personalization of workload prediction models.
- 3) We conduct extensive trace-driven experiments with real-world workloads from multiple cloud providers. We also implement our framework over a joint-cloud computing platform to evaluate its effectiveness in actuality. Compared with the state-of-the-art, our approach improves the workload prediction accuracy by 57.7% in average over all the cloud providers.

To our best knowledge, this is the first privacy-preserving cloud workload prediction framework that comprehensively tackles both data heterogeneity and model heterogeneity across cloud providers. The rest of the paper is organized as follows. Section II presents related works. Section III presents our experimental studies. Section IV presents the details of our proposed framework. Section V presents performance evaluation results. Section VI concludes the paper with future work.

II. RELATED WORK

A. Traditional Cloud Workload Prediction

Calheiros et al. [5] proposed a method that utilizes the ARIMA model for cloud workload forecasting. Peng et al. [8] developed a system that utilizes the logs and meta-data files of cloud applications for application layer traffic forecasting. Mengistu et al. [6] proposed a cloud workload prediction model based on multi-state semi-Markov process. Rossi et al. [7] proposed to utilize Bayesian neural networks and deep learning models for workload prediction, which considers the uncertainty of workloads. Lu et al. [12] proposed a Cloud workload prediction framework based on an improved Gated Recurrent Unit (GRU) neural network, which exploits a resource straggler detection module to ensure the reliability of prediction results. Kim et al. [4] proposed an ensemble model combining multiple predictors (e.g., Linear Regression) for accurate workload prediction. However, due to the limited data generated within a single cloud provider, these methods are incapable of handling complex workloads composed of services and various features from world-wide users.

B. Data Heterogeneity in FL

To deal with model performance degradation caused by data heterogeneity, Shin et al. [41] proposed to select more “information-rich” data as training samples. Tang et al. [42] proposed VHL, which not only addresses Non-IID via virtual homogeneous dataset generation, but also improves data privacy protection. Eichner et al. [38] proposed to train client-specific models considering Non-IID data caused by different cyclic patterns. Liu et al. [39] proposed to utilize the sparsity of CNNs for privacy-preserving similarity measurement of clients’ data. Smith et al. [40] proposed a FL framework to personalize models for clients with Non-IID data. However, these methods are mostly for data with spatial features, and thus cannot be applied to cloud workload time series data, of which temporal features are crucial for model performance.

C. Model Heterogeneity in FL

Li et al. [31] designed a KD based FL approach for clients with heterogeneous model structures. Chen et al. [32] proposed to utilize KD to improve the performance of global and local models with highly heterogeneous model parameters. Afonin and Karimireddy [33] proposed a two-agent kernel regression method to improve model-agnostic FL based on KD. Gong et al. [34] proposed FedKD, which utilizes ensemble KD for model fusion of heterogeneous local models via sharing a public dataset among clients. Xia et al. [35] designed a FL framework that combines mutual learning with KD to address model heterogeneity. Shen et al. [36] proposed a FL framework that utilizes Mutual Learning to deal with data heterogeneity and model heterogeneity. Zhu et al. [43] proposed a data-free knowledge distillation approach to address heterogeneous FL, where the server learns a lightweight generator to ensemble user information in a data-free manner, and the clients use the learned knowledge as an inductive bias. Jeong et al. [44] combined GAN and KD to solve the problem of model

heterogeneity and data heterogeneity. However, these works require clients to send additional information for KD, which significantly increases communication cost and privacy leakage risk [37].

III. EXPERIMENTAL STUDIES AND MOTIVATIONS

In this section, we present the details of our experimental studies and the observations that motivate our system design. The datasets used in these studies are:

- **Alibaba dataset** [45]. It is collected from a large production cluster with over 6,500 GPUs (on 1,800 machines) in Alibaba data centers, spanning from July to August of 2020. It contains the logs of training and inference jobs running ML algorithms, which record workload information such as `cpu_usage`, `plan_cpu`, etc.
- **HPC dataset**. It is collected from three high performance computing cloud service centers for academic research, spanning from September to October of 2022. It contains the logs of scientific computing, large-scale optimization or ML algorithms, which record workload information such as `cpu_usage`, `req_cpu`, etc.
- **Google dataset** [46]. It is collected from Google data center cells with over 52,000 machines, which covers 1-month time frame in 2011 and 2019. It contains the logs of Google’s production workloads, which record workload information such as `Time`, `JobID`, etc.
- **Microsoft dataset** [47]. It is collected from Microsoft’s internal Philly clusters with over hundreds of machines equipped with thousands of GPUs, spanning from August to December of 2017. It contains the logs of deep learning production workloads, which record workload information such as `gpu_util`, `cpu_util`, etc.

A. Heterogeneity of Cloud Workloads

To illustrate the high heterogeneity of workloads from different cloud providers, we first process the aforementioned workload datasets by their features. By saying features, we mean the information tags that each cloud provider uses for differentiating its resources (e.g., `cpu_usage`). Note that some cloud providers use different tags to represent the same feature. This is primarily because that the cloud providers focus on different aspects of the resources. For example, *Microsoft* uses 8 features to represent GPU utilization (i.e., `gpu_0_util`, `gpu_1_util`, ...), while *Alibaba* only uses one feature: `gpu_wrk_util`. Therefore, we apply some preliminary processing on the features, such as synonym substitution, merging of identical features, etc. Finally, the workload features under different cloud providers are summarized in Figure 1. Specifically, the histogram on the left represents the total number of features in each cloud provider. The point matrix on the bottom right represents the combinations of cloud providers that share at least one common feature, where the black points and gray points represent a cloud provider is in or not in a specific combination, respectively. The histogram on the upper right represents the number of features uniquely shared among the corresponding combinations of the cloud providers. For example, *Alibaba* and *HPC* have 29 and 27

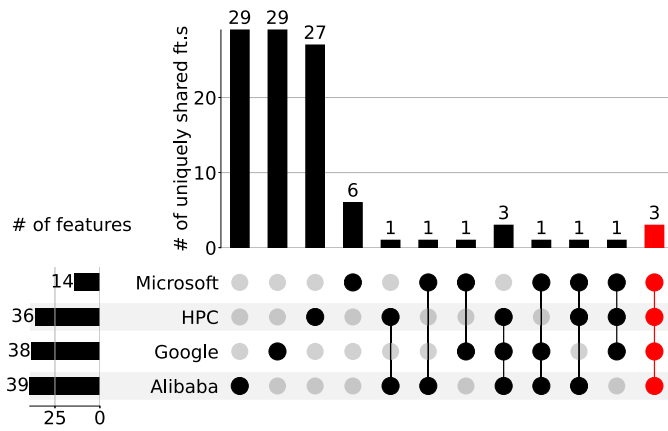


Fig. 1. Summary of workload features under different cloud providers.

unique features, respectively, but they only have 1 uniquely shared feature, which is not covered in the features of the other cloud providers. We can see that the four cloud providers only have 3 features in common, which are CPU utilization, Memory utilization and Time. Thus, we conclude that due to inconsistent system implementations (e.g., infrastructure, service offering, etc.), different cloud providers have quite heterogeneous workload features, which will lead to their various preferences for the optimal workload prediction models (i.e., model heterogeneity). In Section IV-C, we will elaborate how we personalize the local model of each cloud provider according to their respective workload features.

With the extracted workload time series data, we further handle missing and irregular timestamps, and analyze the temporal patterns of several workload features from different cloud providers. Specifically, we first detect and handle irregular timestamps in the workload time series. For normally distributed features, we utilize the Grubbs outlier detection technique [48] to detect irregular timestamps, and utilize the Interquartile Range outlier detection technique [49] otherwise. Then, as in [50], we use the Cubic Spline Interpolation and Expectation Maximization to estimate the real workload values of missing timestamps and the detected irregular timestamps. Figure 2 illustrates the partial time series of CPU utilization and memory utilization in the four cloud providers. We can see that neither memory workloads nor CPU workloads follow regular patterns, and the workloads collected from different cloud providers have heterogeneous temporal characteristics (i.e., data heterogeneity). This motivates us to design new methods for solving the data heterogeneity issues.

B. Necessity of Temporal Features to Virtual Workload Data Generation

VHL has been confirmed effective in solving the Non-IID issue with improved convergence speed, and high generalization performance on both training and testing datasets [42]. It relies on the GAN to generate synthetic virtual data, and utilizes feature calibration to adjust the differences of feature distributions between the virtual data and actual data. However, since VHL cannot accurately calibrate the impact of temporal features, which are dominant and diverse in workload

time series as reflected in Figure 2, it may be inapplicable for the generation of virtual workload data.

To confirm this conjecture, we apply the vanilla GAN [51] and VHL to the generation of virtual workload data, and compare their generated results against actual workloads. Specifically, we first obtain the workload time series data under each feature of the cloud providers. Then, we construct a public noise dataset by sampling on the pure Gaussian noise covering all the features of the cloud providers [42]. Subsequently, we use both the cloud providers' local dataset and the public noise dataset to train the GAN model and the VHL model. Finally, we use the trained models to generate virtual workload data.

Figure 3 illustrates the actual memory utilization time series of *Alibaba* in three days and the virtual workload data generated by the methods. We can see that the actual memory utilization has obvious periodic patterns that drop to valley points at around the 1st time slot, and reach peak points at around the 8th time slot per day. However, the virtual workload data generated by GAN and VHL do not exhibit such patterns. We further conduct dimension reduction techniques, including the Principal Component Analysis (PCA) [52] and the t-SNE (t-distributed Stochastic Neighbor Embedding) [53] on the actual workload data and the generated virtual data to more comprehensively analyze their difference. Specifically, PCA and t-SNE can project high-dimensional data points to lower-dimensional data points while preserving as much of the data's variation as possible, so as to better observe the difference of the major feature distributions of different time-series data. We reduce the dimensionality of actual data and virtual data to a 2-dimensional space for comparison. The results obtained with PCA and t-SNE are illustrated in Figure 4 and Figure 5, respectively. We can see that the distribution of the virtual data generated by GAN is belt-shaped, and conspicuously deviates from that of the actual data either in Figure 4 or Figure 5. In comparison, the distribution of the virtual data generated by VHL is much more similar to that of the actual data. This is primarily due to the effectiveness of VHL's feature calibration in learning the feature distribution from the actual data, so that the gap between the virtual data and the actual data is smaller. However, the results of VHL still severely deviate from the actual data. This is because that the feature calibration in VHL treats all features equally. Considering that temporal features are far more important than the other features for time series data, the incapability of VHL in calibrating the impact of temporal features results in its deviation of temporal feature distribution in virtual data generation. Driven by these observations, we propose a temporal-aware virtual workload dataset generation method, of which details will be elaborated in Section IV-A.

C. Relation Between Planned and Actual Usage of Resources

Under the widely accepted pay-as-you-go pricing models, the users of cloud services need to specify their resource usage plans upon the submission of their jobs [2], [13]. Intuitively, as these plans reflect the amounts of resources expected by the users, they may be helpful for predicting the actual amount of resources that the users will consume. To verify our conjecture,

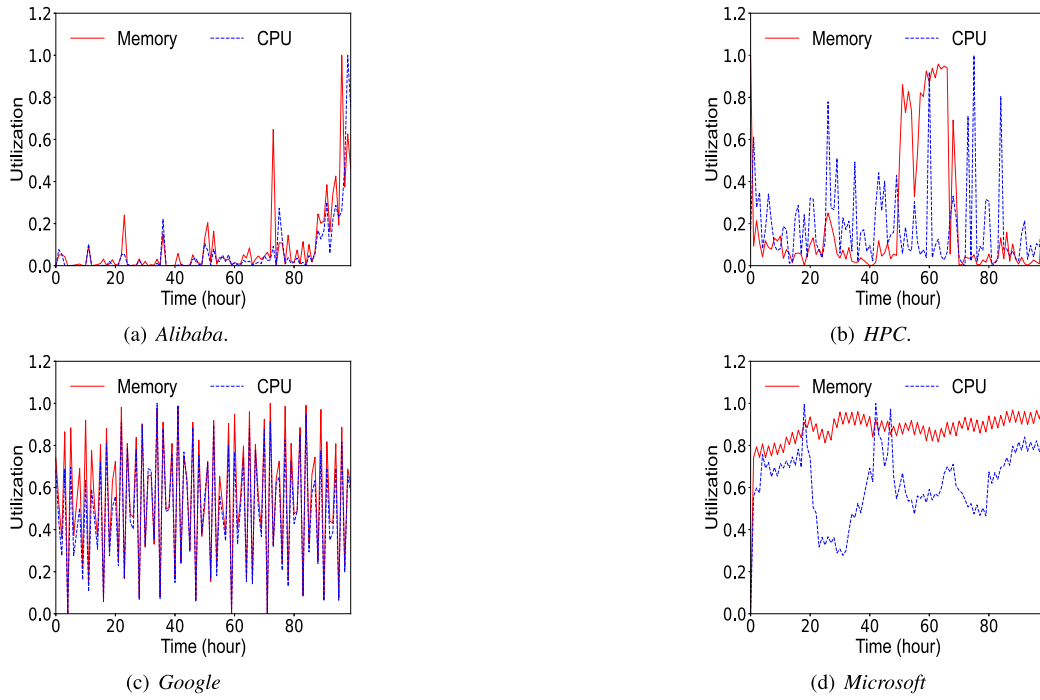


Fig. 2. Workloads of memory and CPU in four cloud providers.

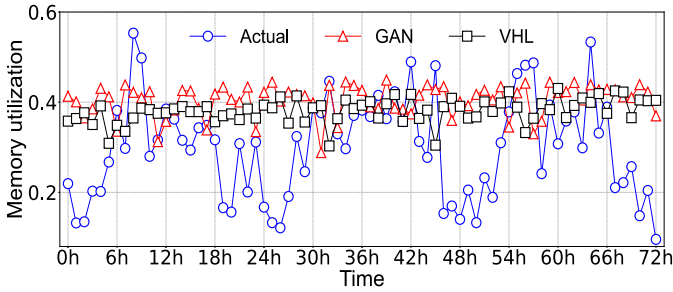


Fig. 3. Actual memory utilization time series in 3 days and virtual workload data generated by different methods.

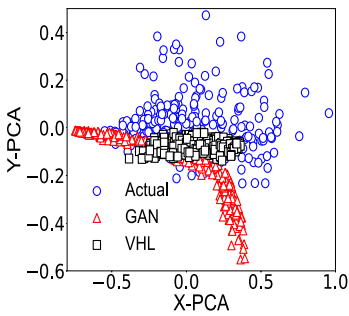


Fig. 4. Dimension reduction results of PCA under different methods.

we aim to analyze the relation between users' planned usage of resources and their actual usage of resources in this section.

Specifically, we use the jobs submitted to the *Alibaba* data centers between July and August of 2020 for this analysis. For each job, we focus on 6 features related to its planned resource usage (i.e., *plan_cpu*, *plan_gpu* and *plan_mem*) and its actual resource usage (i.e., *use_cpu*, *use_gpu* and *use_mem*). We first normalize each feature to be within the same scale (i.e., $[0,1]$) by applying the Min-Max normalization [54], [55]. Then, we prepare the sequences of planned and used

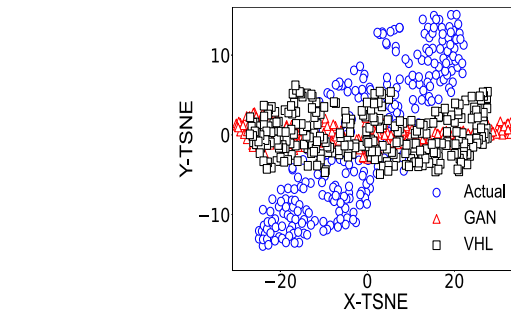


Fig. 5. Dimension reduction results of t-SNE under different methods.

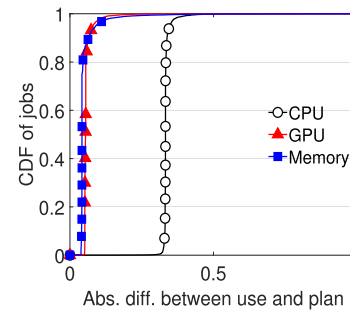


Fig. 6. Absolute differences between planned and actual usage of resources.

resource amounts under each feature (called feature sequence). For example, given the information of Job 1: {*use_cpu* : 0.8, *plan_cpu* : 0.9}, Job 2: {*use_cpu* : 0.3, *plan_cpu* : 0.7} and Job 3: {*use_cpu* : 0.5, *plan_cpu* : 0.6}, the feature sequences corresponding to *use_cpu* and *plan_cpu* are $[0.8, 0.3, 0.5]$ and $[0.9, 0.7, 0.6]$, respectively. Finally, we calculate the element-wise absolute difference between the planned usage feature sequence and the actual usage feature sequence of CPU, GPU and memory, respectively, of which results are illustrated in Figure 6. We can see that except for a

TABLE I

CORRELATION BETWEEN PLANNED AND ACTUAL RESOURCE USAGE

Feature pair	Correlation coefficients
plan_cpu and use_cpu	0.5
plan_gpu and use_gpu	0.62
plan_mem and use_mem	0.5

few jobs with extremely large or small differences, which only takes up $< 2\%$ of the overall jobs, the difference between the planned and actual usage of most jobs concentrates at around 0.05 for Memory, 0.06 for GPU and 0.3 for CPU, respectively. This means that for most jobs, the difference between the planned and actual usage of the resources is very static.

Considering that most of the failed jobs are terminated early after submission (e.g., killed due to programming errors) [2], [13], the jobs that completed successfully are the major source of workloads. Thus, we further measure the correlation between users' planned usage of resources and their actual usage of resources in successful jobs. Specifically, we first extract the jobs with a completion status of *successful*. Then, we calculate the Pearson correlation coefficient [56] between each pair of planned resource usage feature and actual resource usage feature. The Pearson correlation coefficient ranges between -1 and +1, with a negative value indicating negative correlation, and a positive value indicating positive correlation between two feature sequences. The larger the absolute value of the correlation coefficient is, the more correlated the two feature sequences are. Table I shows the calculated pairwise correlation coefficients. We can see that the correlation coefficients between resource usage plan features and their corresponding actual usage features are: 0.5 between plan_cpu and use_cpu, 0.62 between plan_gpu and use_gpu, and 0.5 between plan_mem and use_mem. This means that the users' planned usage of resources and their actual usage of resources are positively correlated. That is, besides the diverse temporal features of workloads, additional consideration of the features related to users' planned usage of resources might be helpful to improving the accuracy of workload prediction models. In Section IV-A, we will elaborate how the features are taken into account in our proposed virtual data generation method.

IV. SYSTEM DESIGN

We focus on the collaborative training of a workload prediction model across K cloud providers (clients), which have highly Non-IID workload datasets and heterogeneous preferences for model structures. Our proposed framework consists of the following three stages as shown in Figure 7:

Stage 1: Virtual Workload Dataset Generation: (Section IV-A). First, we construct the *IID Public Noise* covering the features of all the cloud providers and send it to each cloud provider. Then, based on the *IID Public Noise* and the actual *Workload Dataset*, which includes the features related to *Resource Usage Plan* and *Temporal Patterns*, each cloud provider utilizes *Temporal-aware Feature Calibration* and GAN to generate its *Virtual Dataset*.

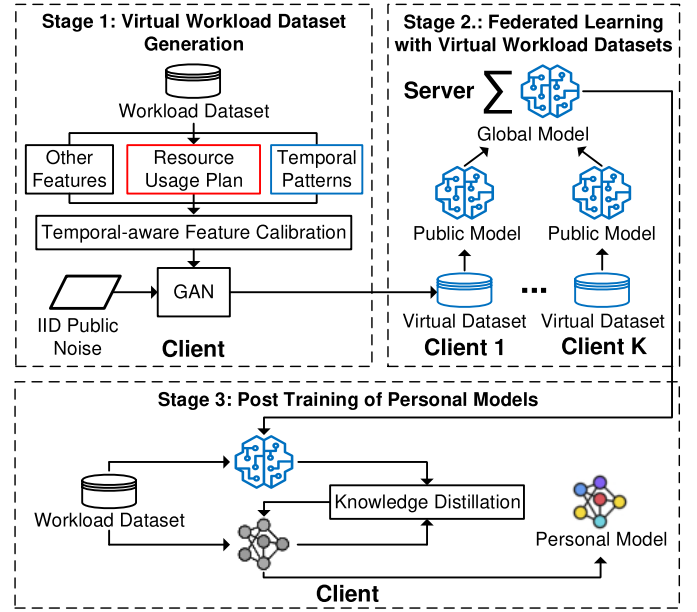


Fig. 7. Framework of our proposed approach.

Stage 2: Federated Learning with Virtual Workload Datasets: (Section IV-B). For FL training, we propose a hybrid local model composed of a homogeneous *Public Model* and a heterogeneous *Personal Model*. Based on the generated *Virtual Datasets*, we let a server schedule the FL training of a *Global Model* across the cloud providers with their *Public Models*.

Stage 3: Post Training of Personal Models: (Section IV-C). For each cloud provider, we utilize *Knowledge Distillation* to transfer knowledge from the trained global *Public Model* to the *Personal Model*, and meanwhile use the private *Workload Dataset* to adjust the training direction of the *Personal Model*.

A. Virtual Workload Dataset Generation

According to Section III-A, the workload data in different cloud providers is highly heterogeneous, which may greatly hinder the FL training of the workload prediction model. A promising solution is to rectify the heterogeneous workload data via the privacy-preserving generation of virtual data for FL training, which is called the VHL [42]. Specifically, VHL is derived from GAN, and uses feature calibration to minimize the distribution discrepancy between the actual dataset and the virtual dataset, which is generated from pure noise (e.g., Gaussian noise). To achieve this goal, VHL formulates a two-player minimax game between the virtual data generator network ω_g and the data discriminator network ω_d with below value function:

$$\begin{aligned} \min_{\omega_g} \max_{\omega_d} V(\omega_d, \omega_g) \\ = \mathbb{E}_{x \sim P_k} [l(f_k(x), y)] + \mathbb{E}_{x \sim P_v} [l(f_k(x), y)] \\ + \lambda \mathbb{E}_y [d(P_k(\varphi(x)|y), P_v(\varphi(x)|y))], \end{aligned} \quad (1)$$

where $f_k(x)$ represents the local model of the k -th cloud provider (client), and $l(f_k(x), y)$ represents the loss function of the local model with x as the input features and y as the output label (e.g., classification task). $\mathbb{E}_{x \sim P_k} [\cdot]$ and $\mathbb{E}_{x \sim P_v} [\cdot]$

represent the expected values of the loss function if x is assumed to follow the distributions of the actual data (P_k) and the virtual data (P_v), respectively. Different from GAN, VHL incorporates a conditional distribution mismatch penalty term ($\lambda \mathbb{E}_y[d(P_k(\varphi(x)|y), P_v(\varphi(x)|y))]$) for feature calibration, where λ is a hyperparameter, $\varphi(x)$ is the feature extractor that calibrates the distribution difference between P_k and P_v [57], and $d(\cdot)$ is a measurement metric of distribution discrepancy (e.g., Wasserstein distance). The ω_g network tries to minimize $V(\omega_d, \omega_g)$, while the ω_d network tries to maximize $V(\omega_d, \omega_g)$, both of which take into account the distribution discrepancy between P_k and P_v .

However, as reflected in Section III-B, when applied to the workload time series data, the virtual data generated by VHL significantly deviates from the actual data. This is because that VHL focuses on classification tasks, which have the same features but different feature distributions. While in our case, the workload data of the cloud providers has both heterogeneous features and feature distributions, especially heterogeneous temporal features.

To take into account the temporal features of the workloads, we propose to label the input features with their corresponding timestamps in feature calibration (called temporal-aware feature calibration). Specifically, we use a time interval (e.g., hour) to partition the workload data under each feature into several *slices*. Thus, each slice is labeled with a timestamp t , and the calculation of the penalty term in Equation (1) is conditioned on t instead of y .

Moreover, the analysis results in Section III-C have demonstrated that the additional consideration of users' resource usage plan might help improve the accuracy of the workload prediction model. Therefore, in addition to temporal features and users' actual resource usage features (e.g., `use_cpu`) widely used in traditional cloud workload prediction methods [4], [5], [6], [7], [8], we propose to take into account the features related to users' resource usage plan (e.g., `plan_cpu`) in virtual workload dataset generation.

To address the problem of heterogeneous features and feature distributions, we propose to utilize pure noise (e.g., Gaussian noise) covering all the features of the cloud providers for virtual dataset generation. Specifically, the procedures are:

- (1) The server collects the feature set of each cloud provider, which is publicly accessible via the resource pricing information of respective cloud providers [58], [59], [60].
- (2) The server uses the collected feature sets to form a union feature set and sample on the pure noise, which covers all the features included in the union feature set, to build the public noise dataset.
- (3) The server sends the public noise dataset to each of the cloud providers.
- (4) Each cloud provider uses the public noise dataset to generate its own virtual dataset by Equation (1).

Since the public noise dataset is IID across the cloud providers, the generated virtual datasets contain no private workload data information of the cloud providers and share homogeneous feature space.

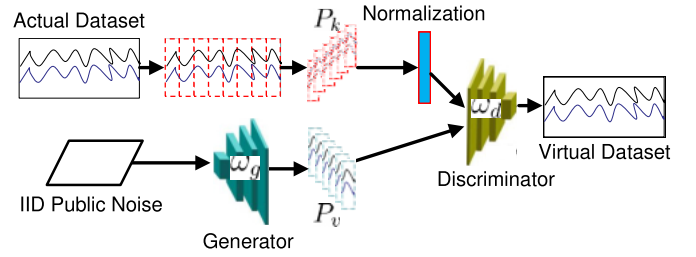


Fig. 8. Workflow of virtual workload data generation.

In summary, the workflow of generating virtual workload datasets for the k -th cloud provider can be illustrated as in Figure 8. First, we partition the actual workload dataset of all the features into slices (i.e., P_k). Since different cloud providers have heterogeneous feature formats as demonstrated in Section III-C, we add a normalization layer before the ω_d network, so that the P_k from different cloud providers covers the same features. Then, by using a pure noise (e.g., Gaussian noise) as the foundation, which is distributed to each cloud provider before training, the ω_g network outputs a temporary dataset P_v that aims to minimize $V(\omega_d, \omega_g)$ by Equation (1). Finally, the ω_d network takes both P_k and P_v , and outputs the virtual dataset of the k -th cloud provider, which maximizes $V(\omega_d, \omega_g)$, and shares the same feature space across the cloud providers.

B. Federated Learning With Virtual Workload Datasets

After Section IV-A, each cloud provider has obtained a virtual workload dataset sharing the same feature space. However, due to the different model preferences of the cloud providers as demonstrated in section III-A, the workload prediction models on different cloud providers will be heterogeneous. To realize the collaborative training of a global workload prediction model across the cloud providers, and meanwhile adaptively personalize the model according to respective preferences of the cloud providers, we propose a hybrid local model composed of a public model and a personal model, which belong to the same model type but have different structures:

- **Public Model** is obtained via the FL training, and shares a homogeneous model structure across different cloud providers. It serves as the cloud providers' proxy in sharing the knowledge of workload features reflected in the virtual datasets across different cloud providers. Considering that FL training is time-consuming and will result in high communication overhead for transferring model parameters, we let the server select a time series prediction model (e.g., GRU neural network) with a small-scale, lightweight structure as the public model.
- **Personal Model** is trained with each cloud provider's private workload data after the completion of the FL training, and serves as the adaptor in combining the knowledge learned in the FL training of the public model with the knowledge reflected in the private actual workload data of the cloud providers. Following the model type selected by the server, each cloud provider specifies a different model structure (e.g., number of

layers, number of hidden units) to adapt to its own workload environment. In Section IV-C, we will elaborate the details of our proposed method for the post training of personal models based on KD.

Since the virtual datasets share the same feature space, the collaborative training of the global public model can be accomplished by applying various popular FL frameworks (e.g., FedAvg [21]) as illustrated in “Stage 2” of Figure 7.

C. Post Training of Personal Models

KD-based FL frameworks will result in high communication cost and risk of privacy leakage due to the sharing of public knowledge among all the clients during each round of FL training [35], [37], which makes KD inappropriate for solving model heterogeneity in traditional FL frameworks. Considering that in our case, the public model will include the knowledge learned from all cloud providers upon the completion of FL training, there is no need to share public knowledge during the training of the public model. Instead, in the post training of the personal model, we utilize KD to transfer knowledge from the trained global public model to the personal model, which not only significantly reduces communication cost, but also avoids the risk of privacy leakage.

Suppose the public model is denoted as ω^* , and the personal model of the k -th cloud provider is denoted as ω'_k . Generally, KD enables the student model (i.e., personal model in our case) to mimic the teacher model’s behavior (i.e., public model in our case) via the formulation of a distillation loss, which captures the difference between the logits of the student and teacher models. However, we do not expect the private model to fully mimic the public model’s predictions as it needs to be personalized with the cloud provider’s private workload data. Considering that the logit output of ω^* is the source of public workload prediction knowledge [61], we propose to combine it with the loss function in the training of the personal model. Thus, we can distill the public workload prediction knowledge from ω^* , and meanwhile use the private workload data to adjust the training direction of ω'_k .

Specifically, considering that Mean Squared Error (MSE) is generally the default loss function for time series prediction [7], we use it as the loss function (denoted by $L_{\omega'_k}$) in the supervised training of ω'_k based on the private workload data of the k -th cloud provider without losing generality:

$$L_{\omega'_k} = \frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2, \quad (2)$$

where y_i is the actual value of the i -th data sample, $\tilde{y}_i$ is the prediction value, and n is the total number of data samples.

Suppose O^* and O'_k represent the logit outputs of ω^* and ω'_k , respectively, which have the same size due to the same workload time series data length used for the training of ω^* and ω'_k . To transfer the workload prediction knowledge from ω^* to ω'_k and for consistency with $L_{\omega'_k}$, we use the MSE between O^* and O'_k to define the divergence between the knowledge of ω^* and ω'_k :

$$D_{\text{MSE}}(O^*, O'_k) = \frac{1}{m} \sum_{j=1}^m (o_j^* - o'_{j,k})^2, \quad (3)$$

where o_j^* and $o'_{j,k}$ are the j -th element of O^* and O'_k , respectively, and m is the number of elements in O^* and O'_k .

For cloud providers with heterogeneous workload data, the public knowledge might not be all beneficial [17], [62]. Therefore, we aim to achieve a balance between the learning of public knowledge and local knowledge. Specifically, we set $1 - \mu$ and μ as the weights of $L_{\omega'_k}$ and D_{MSE} , respectively, where $\mu \in [0, 1]$ controls the weight of the public knowledge in training the personal model, and is separately set by individual cloud providers according to their actual workload environment. In Section V-B.3, we will elaborate the impact of μ on workload prediction performance. Finally, we define the loss function for distilling knowledge from ω^* to ω'_k as:

$$L_{\omega'_k}^* = (1 - \mu)L_{\omega'_k} + \mu D_{\text{MSE}}(O^*, O'_k). \quad (4)$$

As a result, by updating the model parameters with the loss function $L_{\omega'_k}^*$ during training, the trained personal model of the k -th cloud provider can not only gain global workload prediction knowledge from the others, but also adapt to its own workload environment. The post-training workflow of the personal model is illustrated in “Stage 3” of Figure 7.

D. Security Analysis

This paper focuses on a cross-silo FL system. We assume that the server will follow the protocol for FL, but is curious about model updates of the clients, while the clients do not trust others except for themselves. In addition, we assume that the contents transferred between the server and the clients may be overheard during FL training. Under this assumption, the previous data heterogeneity and model heterogeneity solutions for FL [32], [33], [34], [35], [36], [38], [39], [40], [41], [42], [43], [44], which either augment image data with generative learning on shared data label distribution, or use proxy datasets during KD, significantly increase communication cost and privacy leakage risk.

In comparison, on the one hand, our proposed method uses virtual datasets instead of clients’ actual private data to participate in the training of the global FL model. Thus, the actual data will not be directly exposed. On the other hand, the KD in our proposed method does not rely on proxy datasets, and is solely conducted on local private data, during which the logit outputs are only transferred between the public model and the client’s personal model. In summary, our proposed approach can provide better privacy protection for clients via the combination of temporal-aware virtual workload dataset generation, FL training of the public model, and a KD-based post training of clients’ personal models.

In summary, shared data is an important issue affecting the security of KD. We protect data privacy by generating virtual data sets to participate in the generation of the global model of federated learning, so that the actual data is not directly exposed, which has significantly better security than other methods. In addition, we reduce our dependence on the unlabeled proxy dataset, KD is performed based on local private data, and Logits only communicates between the two models at the client. Our approach provides better security by combining separated KD(our post-federated learning) with virtual datasets.

V. PERFORMANCE EVALUATION

A. Experimental Settings

1) *Comparison Methods*: We compare our proposed framework with a representative FL framework based on virtual homogeneous data generation (denoted as *VHL*), [42], a representative FL framework for tackling heterogeneity [63] (denoted as *FedProx*), the vanilla *FedAvg* framework [21], and a GRU-based cloud workload predictor (denoted as *GRU*) [12], which serves as the baseline. Specifically, *VHL* utilizes GAN and feature calibration to generate a rectified virtual dataset for each client without leaking private information. *FedProx* introduces a proximal term to the objective function of model aggregation to penalize the deviation between local models and the global model. *FedAvg* utilizes the aggregated model parameters of clients, which are weight-averaged by the clients' data quantity, to update the model of each client. *GRU* exploits an improved GRU neural network integrated with a resource straggler detection module to capture the most useful workload pattern information for prediction. During the experiments, we divide *GRU* into two forms: *GRU-Local*, which solely uses the workload data of a single cloud provider for model training, and *GRU-All*, which uses the workload data of all cloud providers for model training. We divide our proposed framework into three forms: *Ours*, which consists of all the proposed methodologies, *OursNoVir*, which has no virtual workload dataset generation method, and *OursNoKD*, which has no post training based on KD. To fairly evaluate the performance of our proposed framework in generating virtual time series data, we combine FedAvg with a representative GAN-based time series generation method TGAN [64] (denoted as *TGAN+FedAvg*).

2) *Datasets*: We use the datasets introduced in Section III for the experiments. For each cloud provider, we use the first 80% as the training dataset. Since there is no available cross-cloud workload datasets that comprehensively cover various kinds of workload patterns, we extract 20% workload data from each cloud provider to form the testing dataset. We sample on a pure Gaussian noise covering all the features of the cloud providers to form our public noise dataset.

3) *Models*: *VHL* and our proposed virtual workload data generation method are implemented based on GAN [51]. We use FedAvg [21] as the FL training framework. We select GRU network as the model type of public and personal models, of which structures will be elaborated per experiment. For post training, μ is set per experiment.

4) *Implementation*: We conduct evaluations on a Linux server with 2 GeForce RTX 3090 GPUs, 1 Intel Xeon CPU, and 256 GB memory. We implement our framework with PyTorch, and utilize PySyft [65] to encrypt model parameters. The FL clients are *Alibaba*, *Google*, *Microsoft* and *HPC*.

B. Experimental Results

1) *Comparison of Workload Prediction Accuracy*: In this section, we evaluate the performance of different methods on workload prediction accuracy. Specifically, the generation of virtual workload datasets is completed before FL training. We first distribute the public Gaussian noise to each of the

TABLE II
PERSONAL MODEL STRUCTURES OF CLOUD PROVIDERS

	<i>Alibaba</i>	<i>Google</i>	<i>Microsoft</i>	<i>HPC</i>
Number of hidden layers	4	2	4	2
Number of hidden units	512	512	256	256

TABLE III
MSES OF PREDICTION RESULTS ON CPU UTILIZATION, GPU UTILIZATION AND MEMORY UTILIZATION (LOWER IS BETTER)

	<i>Alibaba</i>	<i>Google</i>	<i>Microsoft</i>	<i>HPC</i>
<i>GRU-Local</i>	0.0618 $\pm$ 0.0399	0.1283 $\pm$ 0.0482	0.0560 $\pm$ 0.0166	0.0533 $\pm$ 0.0230
<i>GRU-All</i>	0.0796 $\pm$ 0.0261	0.1058 $\pm$ 0.0472	0.0681 $\pm$ 0.0416	0.2389 $\pm$ 0.0746
<i>Ours</i>	0.0594 $\pm$ 0.0116	0.0689 $\pm$ 0.0243	0.0384 $\pm$ 0.0082	0.0508 $\pm$ 0.0199
<i>OursNoVir</i>	0.0931 $\pm$ 0.0668	0.1166 $\pm$ 0.0519	0.1062 $\pm$ 0.0412	0.1241 $\pm$ 0.1059
<i>OursNoKD</i>	0.0647 $\pm$ 0.0723	0.0647 $\pm$ 0.0723	0.0647 $\pm$ 0.0723	0.0647 $\pm$ 0.0723
<i>VHL</i>	0.1284 $\pm$ 0.1063	0.1284 $\pm$ 0.1063	0.1284 $\pm$ 0.1063	0.1284 $\pm$ 0.1063
<i>TGAN+FedAvg</i>	0.1400 $\pm$ 0.1041	0.1400 $\pm$ 0.1041	0.1400 $\pm$ 0.1041	0.1400 $\pm$ 0.1041
<i>FedAvg</i>	0.1514 $\pm$ 0.1040	0.1514 $\pm$ 0.1040	0.1514 $\pm$ 0.1040	0.1514 $\pm$ 0.1040
<i>FedProx</i>	0.0853 $\pm$ 0.1214	0.0853 $\pm$ 0.1214	0.0853 $\pm$ 0.1214	0.0853 $\pm$ 0.1214

cloud providers. Then, we set $\lambda=0.15$ in Equation (1), and train the GAN for 3000 epochs with a batch size of 32. To evaluate the impact of model heterogeneity, we let the cloud providers in *GRU*, *Ours* and *OursNoVir* use GRU networks with different structures as their personal models, of which details are shown in Table II. However, since the training workload data in *GRU* is isolated in different cloud providers, the model training in *GRU* is separately conducted for each cloud provider, which solely uses the workload data of the cloud provider. Since the FL training of *OursNoKD*, *VHL*, *TGAN+FedAvg*, *FedProx* and *FedAvg* does not consider model personalization, we use a GRU network with 2 hidden layers, each of which has 256 hidden units, as the public (global) model. All the GRU networks have one input layer with the size of 6, and one output layer with the size of 3. The FL training is conducted for 30 epochs. In post training, μ is set to 0.6, the length of observation time step is 2, and the post training is conducted for 30 epochs. The learning rate is 0.001 for all the trainings. For each cloud provider, based on the testing dataset composed of workload patterns from the four cloud providers, we measure the MSEs of prediction results on CPU utilization, GPU utilization and memory utilization by Equation (2). The mean and standard deviation of the MSEs are illustrated as “mean $\pm$ standard deviation” in Table III. Over all the cloud providers, the results generally follow: *Ours* < *GRU-Local* < *OursNoKD* < *GRU-All* < *FedProx* < *OursNoVir* < *VHL* < *TGAN+FedAvg* < *FedAvg*.

We can see that the MSEs of *OursNoKD*, *VHL*, *TGAN+FedAvg*, *FedProx* and *FedAvg* remain the same over different cloud providers. This is because these methods do not consider model personalization, so the cloud providers

can only use the trained public (global) model for workload prediction. *FedAvg*, *FedProx*, and *TGAN+FedAvg* result in similar prediction MSEs, which are much higher than the other methods. This is primarily because that the workload patterns from different cloud providers are highly heterogeneous, while the prediction models trained in *FedAvg* and *FedProx* are not personalized according to the heterogeneity degree of local data, and *TGAN+FedAvg* cannot incorporate the heterogeneous workload patterns of the other cloud providers into the generation of workload time series data as well.

In comparison, *OursNoKD* and *VHL* significantly improve the prediction accuracy. This is primarily due to the benefit of feature calibration, which can effectively align the heterogeneous features in virtual training data generation. Moreover, since our proposed temporal-aware feature calibration can further take into account both temporal features and resource usage plan features, *OursNoKD* significantly outperforms *VHL*.

Although *OursNoVir* directly uses the local actual workload dataset for FL training, its performance is better than *VHL* over all the cloud providers. This is mostly due to the model personalization via KD, which enables the cloud providers to alleviate the problems caused by Non-IID workload data through distilling local knowledge. However, since the performance of the public model in *OursNoVir* has already been deteriorated by the Non-IID workload data, the KD procedure cannot bring much useful knowledge to the personal models of the cloud providers, which undermines the overall performance of *OursNoVir*.

It is worth noticing that although *GRU-Local* solely uses the workload data of a single cloud provider for model training, the trained GRU model can achieve the second lowest MSE on almost all the cloud providers. Recall that the testing dataset is composed of the workload patterns from all the four cloud providers, the results of *GRU-Local* demonstrate that the cloud providers have partial workload patterns in common, which enables the isolated GRU models to handle cross-cloud workloads in the testing dataset. However, we can notice that the performance of *GRU* is especially bad on *Google*, which is due to the singular workload patterns of *Google* compared to the other cloud providers. The prediction model solely trained on *Google*'s data has insufficient knowledge on the workload patterns of the other cloud providers. Thus, no matter how much data *Google* has for the training of *GRU*, its workload prediction model cannot achieve comparable performance to that of *Ours*. Meanwhile, we can also notice that even if we use the workload data of all the cloud providers to train the GRU models, the MSEs of *GRU-All* are worse than those of *GRU-Local* on most cloud providers. This means that not all workload data are beneficial to increasing the accuracy of workload prediction models. The transfer of useful knowledge must be tailored according to the actual workload pattern per cloud provider.

In contrast, thanks to the knowledge learned from the FL training, which is conducted on the virtual workload dataset with temporal features and resource usage plan features, and the combination of public knowledge and local knowledge via personalized KD, *Ours* achieves the lowest MSE, and reduces

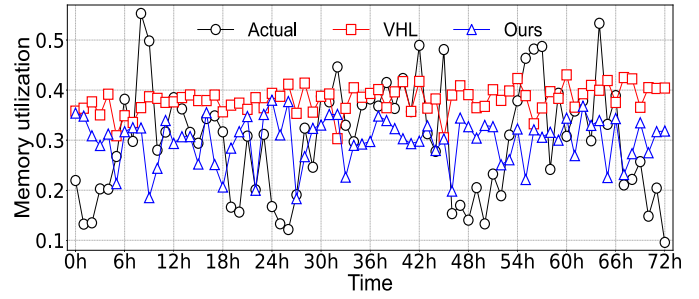


Fig. 9. Comparison of generated virtual workload datasets.

the MSE by 57.7%, 61.2% and 64.1% in average over all the cloud providers in comparison with *VHL*, *TGAN+FedAvg* and *FedAvg*, respectively.

2) *Comparison of Virtual Dataset Generation Performance*: To evaluate the performance of different methods in generating virtual workload datasets, in Figure 9, we intuitively illustrate how well the virtual datasets generated by *Ours*, *VHL* and *TGAN* [66], which is a representative GAN-based time series generation method, fit the actual memory utilization time series of *Alibaba* in three days. We can see that compared with *VHL* and *TGAN*, the virtual workload data generated by *Ours* exhibit much more similar fluctuations and patterns as the actual data.

In addition to the workload data snippet, we further conduct the PCA [52] and the t-SNE [53] on the whole actual workload data of *Alibaba* (denoted as *Actual*), and the virtual datasets generated by *Ours*, *VHL* and *TGAN*. Specifically, Figure 10(a) and Figure 10(b) respectively illustrate the PCA and t-SNE dimensionality reduction results of *Actual*, *Ours* and *VHL*. In Figure 10(a), we can see that the actual data is widely scattered, but the virtual data generated by *VHL* only concentrate in a small area. In comparison, the virtual data generated by *Ours* is more extensively distributed. In Figure 10(b), we can see that the actual data presents a diagonal bag-like distribution, while the distribution of the virtual data generated by *VHL* is almost orthogonal to that of the actual data. In comparison, the distribution of the virtual data generated by *Ours* almost overlaps 2/3 of the actual data.

Figure 10(c) and Figure 10(d) respectively illustrate the PCA and t-SNE dimensionality reduction results of *Actual*, *Ours* and *TGAN*. In Figure 10(c), compared with *Ours*, the data generated by *TGAN* is very concentrated without a good capture of the distribution of the real data. In Figure 10(d), although the data generated by *TGAN* is scattered, it is still much less alike to the actual data distribution when compared with *Ours*. This is because that *Ours* not only considers temporal patterns, such as periodicity, as in *TGAN*, but also considers the distribution of workloads under different timestamps. As a result, our proposed virtual workload generation method can better learn the distribution of the actual data.

Considering that the GAN is trained offline, it needs to be retrained as long as there is unexpected or sudden workload shift. To determine whether the computational overhead of the GAN retraining will impact model training, we measure the time costs of GAN training (1000 epochs), FL (50 epochs)

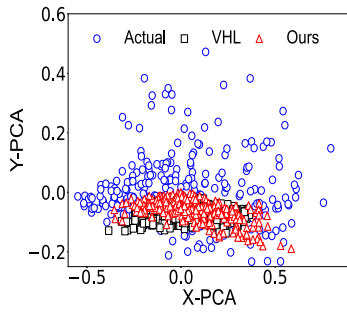
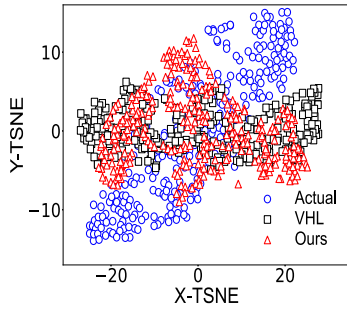
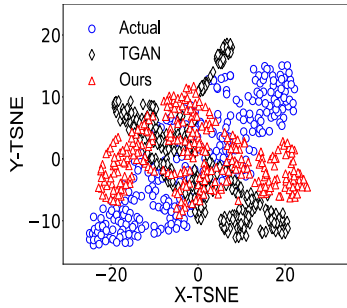
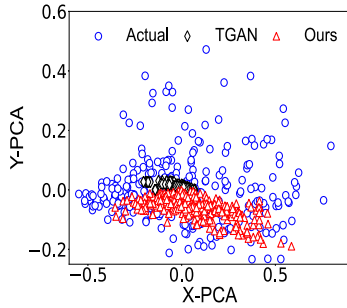
(a) *Ours* versus *VHL* under PCA.(b) *Ours* versus *VHL* under t-SNE.(d) *Ours* versus *TGAN* under t-SNE.

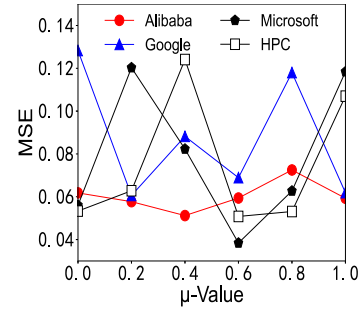
Fig. 10. Dimensionality reduction results of PCA and t-SNE.

and post-training (100 epochs) per client during the overall model training process for each cloud provider in *Ours*. For comparison, we also measure the time costs per client in *FedAvg*. The results averaged by the number of clients are summarized in Table IV. We can see that the most time-consuming operations are FL and post-training, of which time costs are comparable to that of *FedAvg*. In contrast, the time cost of GAN training is negligible. These results demonstrate that the offline retraining overhead of the GAN is trivial, and will not delay the model training progress.

3) *Impact of Model Personalization*: Recall that μ controls the weight of public knowledge in training the personal model (Section IV-C). Considering that *Ours* exhibits inconsistent

TABLE IV
COMPARISON OF TIME COSTS

Method	Time Cost		
	Provider	GAN training (1000 epochs)	FL (50 epochs)
<i>Ours</i>	<i>Alibaba</i>	83.55s	487.31s
	<i>Google</i>	86.20s	487.32s
	<i>HPC</i>	95.72s	487.33s
	<i>Microsoft</i>	85.03s	487.34s
<i>FedAvg</i>		665.65s (50 epochs)	
		Post-training (100 epochs)	

Fig. 11. Change of MSE under various μ values.

prediction performance on different cloud providers, we infer that the cloud providers may have different preferences in distilling the public knowledge. To confirm this conjecture and evaluate the impact of μ on model performance, we vary the μ value from 0 to 1 with a step size of 0.2, and measure the corresponding MSEs of prediction results, which are illustrated in Figure 11.

We can see that along with the increase of μ , the prediction MSEs over all the cloud providers generally follow the “drop-valley-rise” pattern. This means that there exists a μ value that optimally balances the contributions of the public knowledge and local data in Equation (4) for each cloud provider. We can also notice that the optimal μ values vary significantly over different cloud providers, which confirms our conjecture. Specifically, the MSEs of *HPC* and *Microsoft* reach their valley points at $\mu = 0.6$, while *Alibaba* reaches its MSE valley point at $\mu = 0.4$. However, it is worth noticing that the prediction MSEs of *Google* are almost equally low at both $\mu = 0.2$ and $\mu = 1.0$. This is primarily due to the same reason as explained in Section V-B.1: *Google*’s workload patterns are relatively more unique to the other cloud providers. Given a mixed testing dataset, in which each cloud provider takes up an equal ratio of data samples, *Google*’s personal model can only achieve its best performance via fully distilling the public knowledge ($\mu = 1.0$), which guarantees its good prediction performance on workloads other than *Google*’s, or limiting the impact of public knowledge ($\mu = 0.2$), which guarantees its good prediction performance on its own workloads and a few workloads from the other cloud providers.

Considering that KD temperature controls the shape of the model’s prediction probability distribution, thereby influencing the transfer of knowledge from the teacher, we further analyze the sensitivity of *Ours* to KD temperature. Specifically, in addition to μ , we also vary the temperature from 0.5 to 10, and measure the MSE of the prediction model on the workload of

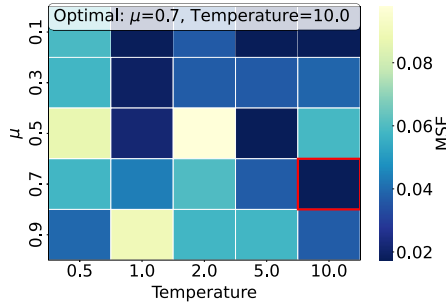
Fig. 12. MSE under various μ and temperature values.

TABLE V

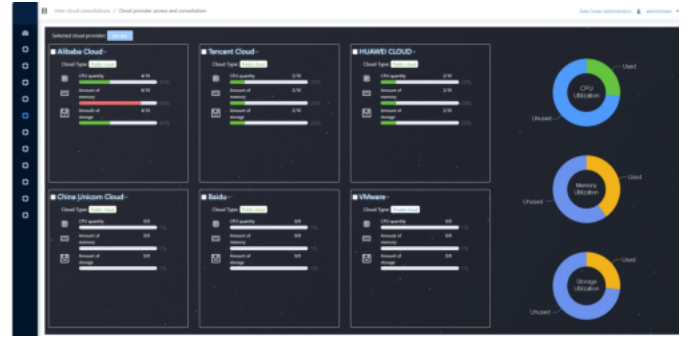
MSES OF WORKLOAD PREDICTION RESULTS WITH AND WITHOUT CONSIDERING RESOURCE USAGE PLAN FEATURES

	RNN	RNN+	GRU	GRU+	LSTM	LSTM+
Memory utilization	0.0285	0.0280	0.0238	0.0229	0.0155	0.0142
CPU utilization	0.0351	0.0321	0.0189	0.0179	0.0200	0.0197
GPU utilization	0.0284	0.0264	0.0107	0.0105	0.0146	0.0145

Alibaba under each combination of μ and temperature values. Figure 12 illustrates the heat map matrix of the measured MSEs. The darker color a square has, the lower MSE that the corresponding combination of μ and temperature values results in. We can see that when $\mu = 0.7$ and the temperature is 10.0 (the square circled with the red frame), the prediction model has the lowest MSE. In conclusion, the optimal selection of μ value and KD temperature of a cloud provider needs to fully consider the heterogeneity degree of the cross-cloud workloads that the provider is about to handle.

4) *Effectiveness of Resource Usage Plan Features in Workload Prediction*: To illustrate the effectiveness of resource usage plan features in improving workload prediction accuracy, we further measure the performance of RNN, GRU and LSTM on various resources with and without taking into account the features, respectively. Specifically, to exclude the impact of data heterogeneity and model heterogeneity, we conduct FL model training with one client based on the memory utilization, CPU utilization and GPU utilization datasets of *Alibaba*. The FL training is conducted for 100 epochs. The GRU network settings are the same as in Section V-B.1. The RNN network has 1 SimpleRNN layer, which has 128 units. The LSTM network has 2 LSTM layers, each of which has 50 units. The output layer size of all the networks is 1, while the input size of these networks varies depending on the number of input features and the KD observation time step. The measured MSEs of the prediction results are illustrated in Table V. RNN+, GRU+, and LSTM+ represent the counterparts that take into account the resource usage plan features, respectively.

We can see that the MSEs of RNN+, GRU+, and LSTM+ have 0.6% - 8.5% improvement compared with RNN, GRU, and LSTM. Considering that RNN+, GRU+, and LSTM+ do not enjoy the knowledge learned via the KD-based post training of clients' personal model, we conclude that the



(a) Status page of connected cloud providers.



(b) Workload prediction results of respective cloud providers.

Fig. 13. Real-world implementation of the proposed framework over a joint-cloud computing platform.

additional consideration of resource usage plan features can effectively improve the accuracy of workload prediction.

C. Real-World Implementation

To measure the effectiveness of the proposed framework in actuality, we further carry out its real-world implementation over a joint-cloud computing platform [67], which provides centralized resource monitoring and management of multiple cloud providers including *Alibaba*, *Huawei*, *China Unicom*, *Tencent*, etc. The platform is based on the SaCa Aclome Operation Center of *Neusoft* [68], which is built with the Java-based mainstream framework *Spring Cloud* [69], and uses *PostgreSQL* [70] as the database, *Redis* [71] as the middleware, and *Vue* [72] as the front-end page. The platform realizes the access and management of the connected cloud providers through their elastic Cloud Computing instances. For the connected cloud providers, the platform cyclically collects their cloud resource status information, such as resource types, resource usage, etc., through accessing their SDK or OpenAPI. Figure 13(a) shows the status page of the cloud providers connected to the platform, which illustrates the current status information of multiple computing resources such as CPU utilization, Memory utilization, Storage utilization, etc.

1) *Experiment Settings*: The implementation environment follows a “coordinator/cloud service provider” architecture. We select a physical machine from the resource scheduling system of each connected cloud provider, which serves as the clients in the FL training of the public model and the post training of the personal models. The training workload data

TABLE VI

MSEs OF PREDICTION RESULTS ON CPU UTILIZATION, GPU UTILIZATION AND MEMORY UTILIZATION (LOWER IS BETTER)

	<i>Alibaba</i>	<i>Huawei</i>	<i>Tencent</i>
<i>GRU</i>	0.0470 $\pm$ 0.0247	0.0511 $\pm$ 0.0351	0.0504 $\pm$ 0.0389
<i>Ours</i>	0.0470 $\pm$ 0.0061	0.0451 $\pm$ 0.0205	0.0308 $\pm$ 0.0088

of each cloud provider is processed from its historical status information records, while the target workload data for prediction comes from the platform users who might deploy various tasks (e.g., ML tasks, HPC tasks, video streaming tasks). We set the number of FL training rounds to 30, and the number of post training rounds to 50. The hyperparameter μ of all the cloud providers is uniformly set to 0.1. To guarantee the stable transfer of model parameters, we use the TCP protocol for communication as in previous FL implementations [73], [74], [75]. In addition, we let the coordinator server create a separate thread for each client to improve the training efficiency of the global model, and interlace the thread execution rounds to ensure the synchronization of the clients' training progress. Moreover, we develop an integrity verification mechanism based on the MD5 code [76], [77] to ensure the integrity of the parameters received at the coordinator server end.

2) *Experiment Results*: Similar to the trace-driven experiments, we also use the *GRU* method, which solely uses the actual workload data of a single cloud provider for model training, as the baseline. Figure 13(b) shows the workload prediction results of partial cloud providers: *Alibaba*, *Huawei* and *Tencent*. The MSEs of the prediction results on different cloud providers are summarized in Table VI. We can see that the performance of *Ours* is significantly better than that of *GRU* on *Huawei* and *Tencent*, but exhibits little improvement on *Alibaba*. This is primarily because that the historical workloads in *Huawei* and *Tencent* are mostly dominated by HPC tasks and ML tasks, so they can enjoy more prediction improvement via learning from the other cloud providers in *Ours*. However, the historical workloads in *Alibaba* are already mixed with various kinds of tasks, so the performance improvement it can harvest from *Ours* is quite limited.

VI. CONCLUSION

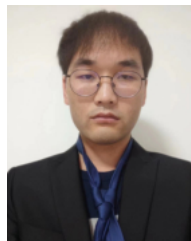
Our proposed framework is the first to achieve privacy-preserving cross-cloud workload prediction with comprehensive solutions to both data heterogeneity and model heterogeneity of cloud providers. To cope with heterogeneous workloads without privacy leakage, we first design a GAN based virtual workload dataset generation method that incorporates the temporal and resource-usage-plan-related features of the workloads via temporal-aware feature calibration. To cope with model heterogeneity, we then design a FL approach based on a hybrid local model composed of a lightweight homogeneous public model and a heterogeneous personal model. Finally, we design a KD based method to post-train the personal model with both private workload knowledge and shared workload knowledge learned from the trained global public model. Extensive experiments on various real-world workloads and actual implementations demonstrate that compared with the state-of-the-art, our framework improves

workload prediction accuracy by 57.7% in average over all cloud providers. In future work, we plan to further improve the performance of virtual workload dataset generation, while enhancing the privacy protection of workload data.

REFERENCES

- [1] M. Tirmazi et al., "Borg: The next generation," in *Proc. EuroSys*, 2020, pp. 1–14.
- [2] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, "Analysis of large-scale multi-tenant GPU clusters for DNN training workloads," in *Proc. ATC*, 2019, pp. 947–960.
- [3] I. Stoica and S. Shenker, "From cloud computing to sky computing," in *Proc. Workshop Hot Topics Operating Syst.*, Jun. 2021, pp. 26–32.
- [4] I. K. Kim, W. Wang, Y. Qi, and M. Humphrey, "Forecasting cloud application workloads with CloudInsight for predictive resource management," *IEEE Trans. Cloud Comput.*, vol. 10, no. 3, pp. 1848–1863, Jul. 2022.
- [5] R. N. Calheiros, E. Masoumi, R. Ranjan, and R. Buyya, "Workload prediction using ARIMA model and its impact on cloud applications' QoS," *IEEE Trans. Cloud Comput.*, vol. 3, no. 4, pp. 449–458, Oct. 2015.
- [6] T. M. Mengistu, D. Che, A. Alahmadi, and S. Lu, "Semi-Markov process based reliability and availability prediction for volunteer cloud systems," in *Proc. IEEE 11th Int. Conf. Cloud Comput. (CLOUD)*, Jul. 2018, pp. 359–366.
- [7] A. Rossi, A. Visentin, S. Prestwich, and K. N. Brown, "Bayesian uncertainty modelling for cloud workload prediction," in *Proc. IEEE 15th Int. Conf. Cloud Comput. (CLOUD)*, Jul. 2022, pp. 19–29.
- [8] Y. Peng, K. Chen, G. Wang, W. Bai, Z. Ma, and L. Gu, "HadoopWatch: A first step towards comprehensive traffic forecasting in cloud computing," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2014, pp. 19–27.
- [9] D. Xiao, B. Cao, and W. Wu, "EFL-WP: Federated learning-based workload prediction in inter-cloud environments," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2022, pp. 1–10.
- [10] M. Masdari and A. Khoshnevis, "A survey and classification of the workload forecasting methods in cloud computing," *Cluster Comput.*, vol. 23, no. 4, pp. 2399–2424, Dec. 2020.
- [11] Z. Ding, B. Feng, and C. Jiang, "COIN: A container workload prediction model focusing on common and individual changes in workloads," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4738–4751, Dec. 2022.
- [12] Y. Lu, L. Liu, J. Panneerselvam, B. Yuan, J. Gu, and N. Antonopoulos, "A GRU-based prediction framework for intelligent resource management at cloud data centres in the age of 5G," *IEEE Trans. Cognit. Commun. Netw.*, vol. 6, no. 2, pp. 486–498, Jun. 2020.
- [13] Q. Weng et al., "Mlaas in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters," in *Proc. NSDI*, 2022, pp. 945–960.
- [14] M. Barika, S. Garg, A. Chan, and R. N. Calheiros, "Scheduling algorithms for efficient execution of stream workflow applications in multicloud environments," *IEEE Trans. Services Comput.*, vol. 15, no. 2, pp. 860–875, Mar. 2022.
- [15] P. Mell and T. Grance, "The NIST definition of cloud computing," *Nat. Inst. Sci. Technol., Special Publication*, vol. 800, no. 2011, p. 145, 2011.
- [16] Z. Yang et al., "SkyPilot: An intercloud broker for sky computing," in *Proc. NSDI*, 2023, pp. 437–455.
- [17] P. Kairouz, "Advances and open problems in federated learning," *Found. Trends Machine Learning*, vol. 14, nos. 1–2, 2021, pp. 1–210.
- [18] (2022). 2022 Global Hybrid Cloud Trends Report. [Online]. Available: <https://www.cisco.com/c/en/us/solutions/hybrid-cloud/2022-trends.html>
- [19] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for YouTube recommendations," in *Proc. 10th ACM Conf. Recommender Syst.*, Sep. 2016, pp. 191–198.
- [20] Z. Zhao et al., "Social-aware movie recommendation via multimodal network learning," *IEEE Trans. Multimedia*, vol. 20, no. 2, pp. 430–440, Feb. 2018.
- [21] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2016, pp. 1273–1282.
- [22] J. Liu, Z. Mi, Z. Huang, Z. Hua, and Y. Xia, "Hcloud: A serverless platform for JointCloud computing," in *Proc. IEEE Int. Conf. Joint Cloud Comput.*, Aug. 2020, pp. 86–93.
- [23] S. Duan, D. Zhang, Y. Wang, L. Li, and Y. Zhang, "JointRec: A deep-learning-based joint cloud video recommendation framework for mobile IoT," *IEEE Internet Things J.*, vol. 7, no. 3, pp. 1655–1666, Mar. 2020.

- [24] S. Luo et al., "Characterizing microservice dependency and performance: Alibaba trace analysis," in *Proc. ACM Symp. Cloud Comput.*, Nov. 2021, pp. 412–426.
- [25] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," in *Proc. ICLR*, 2019, pp. 1–26.
- [26] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. ICML*, 2019, pp. 5132–5143.
- [27] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *JMLR*, vol. 20, no. 1, pp. 1–21, 2019.
- [28] P. Ren et al., "A comprehensive survey of neural architecture search: Challenges and solutions," *ACM Comput. Surveys*, vol. 54, no. 4, pp. 1–34, May 2022.
- [29] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM TIST*, vol. 10, no. 2, pp. 1–19, 2019.
- [30] H. Baek et al., "Joint superposition coding and training for federated learning over multi-width neural networks," in *Proc. IEEE Conf. Comput. Commun.*, May 2022, pp. 1729–1738.
- [31] B. Li, Y. Shi, Y. Guo, Q. Kong, and Y. Jiang, "Incentive and knowledge distillation based federated learning for cross-silo applications," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, May 2022, pp. 1–6.
- [32] H. Chen, Johnny, Wang, and H. Vikalo, "The best of both worlds: Accurate global and personalized models through federated learning with data-free hyper-knowledge distillation," in *Proc. ICLR*, 2023, pp. 1–26.
- [33] A. N. Afonin and S. P. Karimireddy, "Towards model agnostic federated learning using knowledge distillation," in *Proc. ICLR*, 2021, pp. 1–23.
- [34] X. Gong et al., "Preserving privacy in federated learning with ensemble cross-domain knowledge distillation," in *Proc. AAAI*, 2022, vol. 36, no. 11, pp. 11891–11899.
- [35] J. Xia, T. Liu, Z. Ling, T. Wang, X. Fu, and M. Chen, "PervasiveFL: Pervasive federated learning for heterogeneous IoT systems," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 11, pp. 4100–4111, Nov. 2022.
- [36] T. Shen et al., "Federated mutual learning," 2020, *arXiv:2006.16765*.
- [37] Y. Wang, W. Fan, K. Yang, N. Alhusaini, and J. Li, "A knowledge distillation-based backdoor attack in federated learning," 2022, *arXiv:2208.06176*.
- [38] H. Eichner, T. Koren, H. B. McMahan, N. Srebro, and K. Talwar, "Semi-cyclic stochastic gradient descent," in *Proc. ICML*, 2019, pp. 1764–1773.
- [39] B. Liu, Y. Guo, and X. Chen, "PFA: Privacy-preserving federated adaptation for effective model personalization," in *Proc. Web Conf.*, Apr. 2021, pp. 923–934.
- [40] V. Smith, C.-K. Chiang, M. Sanjabi, and A. Talwalkar, "Federated multi-task learning," in *Proc. NeurIPS*, vol. 30, 2017, pp. 4427–4437.
- [41] J. Shin, Y. Li, Y. Liu, and S.-J. Lee, "FedBalancer: Data and pace control for efficient federated learning on heterogeneous clients," in *Proc. 20th Annu. Int. Conf. Mobile Syst., Appl. Services*, Jun. 2022, pp. 436–449.
- [42] Z. Tang, Y. Zhang, S. Shi, X. He, B. Han, and X. Chu, "Virtual homogeneity learning: Defending against data heterogeneity in federated learning," in *Proc. ICML*, 2022, pp. 21111–21132.
- [43] Z. Zhu, J. Hong, and J. Zhou, "Data-free knowledge distillation for heterogeneous federated learning," in *Proc. ICML*, vol. 139, 2021, pp. 12878–12889.
- [44] E. Jeong, S. Oh, H. Kim, J. Park, M. Bennis, and S.-L. Kim, "Communication-efficient on-device machine learning: Federated distillation and augmentation under non-IID private data," 2018, *arXiv:1811.11479*.
- [45] (2023). *Alibaba Cluster Data*. [Online]. Available: <https://github.com/alibaba/clusterdata/tree/master>
- [46] (2014). *Google Cluster Data*. [Online]. Available: <https://github.com/google/cluster-data/blob/master>
- [47] (2023). *Microsoft Cluster Data*. [Online]. Available: <https://github.com/msr-fiddle/philly-traces>
- [48] F. E. Grubbs, "Procedures for detecting outlying observations in samples," *Technometrics*, vol. 11, no. 1, pp. 1–21, Feb. 1969.
- [49] P. J. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation," *J. Amer. Stat. Assoc.*, vol. 88, no. 424, p. 1273, Dec. 1993.
- [50] A. Tawakuli, D. Kaiser, and T. Engel, "Experience: Differentiating between isolated and sequence missing data," *J. Data Inf. Qual.*, vol. 15, no. 2, pp. 1–15, Jun. 2023.
- [51] I. Goodfellow et al., "Generative adversarial networks," *Commun. ACM*, vol. 63, no. 11, pp. 139–144, 2020.
- [52] K. Pearson, "On lines and planes of closest fit to systems of points in space," *Phil. Mag.*, vol. 2, no. 11, pp. 559–572, 1901.
- [53] L. V. D. Maaten and G. E. Hinton, "Visualizing data using t-SNE," *JMLR*, vol. 9, no. 86, pp. 2579–2605, 2008.
- [54] L. Huang, J. Qin, Y. Zhou, F. Zhu, L. Liu, and L. Shao, "Normalization techniques in training DNNs: Methodology, analysis and application," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 8, pp. 10173–10196, Aug. 2023.
- [55] Y. Wu and K. He, "Group normalization," in *Proc. ECCV*, vol. 128, 2018, pp. 3–19.
- [56] K. Pearson, "Note on regression and inheritance in the case of two parents," *Proc. Roy. Soc. London*, vol. 58, pp. 240–242, Dec. 1895.
- [57] G. Yaroslav et al., "Domain-adversarial training of neural networks," *JMLR*, vol. 17, no. 1, pp. 1–35, 2016.
- [58] (2023). *Aliyun Api*. [Online]. Available: <https://cn.aliyun.com/>
- [59] (2023). *Microsoft Cloud API*. [Online]. Available: <https://azure.microsoft.com/zh-cn/>
- [60] (2023). *Google Cloud API*. [Online]. Available: <https://cloud.google.com/>
- [61] G. E. Hinton, O. Vinyals, and J. B. Dean, "Distilling the knowledge in a neural network," in *Proc. NeurIPS*, 2015, pp. 1–17.
- [62] K. Hsieh, A. Phanishayee, O. Mutlu, and P. B. Gibbons, "The non-IID data quagmire of decentralized machine learning," in *Proc. ICML*, vol. 1, 2019, pp. 4387–4398.
- [63] L. Tian, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proc. Mach. Learn. Syst.*, vol. 1, pp. 429–450, Mar. 2018.
- [64] X. Li, V. Metsis, H. Wang, and A. H. H. Ngu, "TTS-GAN: A transformer-based time-series generative adversarial network," in *Proc. Artif. Intell. Med.*, 2022, pp. 133–143.
- [65] (2023). *Pysyft*. [Online]. Available: <https://github.com/OpenMined/PySyft>
- [66] J. Yoon, D. Jarrett, and M. V. D. Schaar, "Time-series generative adversarial networks," in *Proc. NeurIPS*, vol. 32, 2019, pp. 5508–5518.
- [67] (2024). *Jointcloud Computing Platform*. [Online]. Available: <https://gitee.com/jointcloudcomputing/joint-cloud-computing>
- [68] (2024). *Saca Aclome Operation Center*. [Online]. Available: <https://www.neusoft.com/Products/Platforms/2432/>
- [69] (2024). *Spring Cloud*. [Online]. Available: <https://spring.io/projects/spring-cloud>
- [70] (2024). *Postgresql*. [Online]. Available: <https://www.postgresql.org>
- [71] (May 2024). *Redis*. [Online]. Available: <https://redis.io>
- [72] (May 2024). *Vue*. [Online]. Available: <https://cn.vuejs.org>
- [73] (2021). *Pygad: An Intuitive Genetic Algorithm Python Library*. [Online]. Available: <https://github.com/ahmedfgad/FederatedLearning>
- [74] (2022). *Blockchain-Based-Federated-Learning*. [Online]. Available: <https://github.com/sylvia969301/Blockchain-based-Federated-Learning/tree/main>
- [75] (May 2022). *Federated-Learning-Pytorch-Socket*. [Online]. Available: <https://github.com/Flash-sheep/Federated-Learning-Pytorch-Socket/tree/main>
- [76] R. Rivest. (1992). *RFC1321: The MD5 Message-Digest Algorithm*. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc1321>
- [77] P. Ora and P. R. Pal, "Data security and integrity in cloud computing based on RSA partial homomorphic and MD5 cryptography," in *Proc. Int. Conf. Comput., Commun. Control*, Sep. 2015, pp. 1–6.



Bin Yang (Member, IEEE) received the B.E. degree in information security from the University of Electronic Science and Technology of China in 2020. He is currently pursuing the M.S. degree with the School of Cyberspace Security, Xi'an Jiaotong University. His research interests include federated learning and intercloud computing.



Li Yan (Member, IEEE) received the B.S. degree in information engineering from Xi'an Jiaotong University, Xi'an, China, in 2010, and the Ph.D. degree in computer science from the University of Virginia, Charlottesville, VA, USA, in 2019.

He was a Post-Doctoral Researcher with the Senseable City Laboratory, Massachusetts Institute of Technology, Cambridge, USA, from 2020 to 2021. He is currently an Associate Professor with the School of Cyber Science and Engineering, Xi'an Jiaotong University, Xi'an, China. His research interests include data-driven cyber-physical systems, big data analytics, and mobile computer networks. He was the Best Transactions Paper Awardee of IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS and the Best-in-Session-Presentation Awardee of INFOCOM'2017.



Huanbo Gao received the M.S. degree in computer science and technology from Dalian University of Technology. He is currently a Senior Software Engineer with the Neusoft Group Corporation Ltd. His main research interests include application of long and short term memory network for anomaly detection and state perception.



Zhuozhao Li (Member, IEEE) received the Ph.D. degree in computer science from the University of Virginia. He was a Post-Doctoral Scholar with the University of Chicago. He is currently an Assistant Professor with the Department of Computer Science and Engineering and Research Institute of Trustworthy Autonomous Systems, Southern University of Science and Technology. His research interests include broad areas of distributed systems, cloud computing, and high-performance computing, with the emphasis on developing working prototypes for

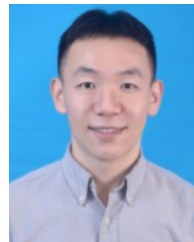
real-world problems and designing the foundation methodologies to optimize system performance for efficient computing.



Xingye Sun received the B.E. degree in software engineering from Dalian Jiaotong University. He is currently a Senior Software Engineer the Neusoft Group Corporation Ltd. His main research interests include cloud computing, virtualization, and DevOps.



Mingjun Kao received the M.S. degree in computer science and technology. He is currently the Product Director of cloud management and operation related products with the Neusoft Group Corporation Ltd. He has in-depth research interests include areas, such as cloud computing, ITOM, and AIOps.



Chao Shen (Fellow, IEEE) was a Research Scholar with Carnegie Mellon University, from 2011 to 2013. He is currently a Professor with the School of Electronic and Information Engineering, Xi'an Jiaotong University, China, where he is also the Associate Dean of the School of Cyber Security. He is also with the Ministry of Education Key Laboratory for Intelligent Networks and Network Security. His research interests include network security, human-computer interaction, insider detection, and behavioral biometrics.